Part I — CIFAR-10 CNN Training — ResNet20

In [80]:
```
import argparse
import os
import time
import shutil

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import torch.backends.cudnn as cudnn



import torchvision
import torchvision.transforms as transforms

from models import *   # bring everything in the folder models

global best_prec
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
use_gpu = torch.cuda.is_available()
print('=> Building model...')


batch_size = 128

model_name = "resnet20"
model = resnet20_cifar()



normalize = transforms.Normalize(mean=[0.491, 0.482, 0.447], std=[0.247, 0.243,


train_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=True,
    download=True,
    transform=transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        normalize,
    ]))
trainloader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_size,


test_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=False,
    download=True,
    transform=transforms.Compose([
        transforms.ToTensor(),
        normalize,
    ]))
```

```
testloader = torch.utils.data.DataLoader(test_dataset, batch_size=batch_size, sh


print_freq = 100 # every 100 batches, accuracy printed. Here, each batch include
# CIFAR10 has 50,000 training data, and 10,000 validation data.

def train(trainloader, model, criterion, optimizer, epoch):
    batch_time = AverageMeter()   ## at the begining of each epoch, this should
    data_time = AverageMeter()
    losses = AverageMeter()
    top1 = AverageMeter()

    model.train()

    end = time.time()  # measure current time

    for i, (input, target) in enumerate(trainloader):
        # measure data loading time
        data_time.update(time.time() - end)  # data loading time

        input, target = input.to(device), target.to(device)

        # compute output
        output = model(input)
        loss = criterion(output, target)

        # measure accuracy and record loss
        prec = accuracy(output, target)[0]
        losses.update(loss.item(), input.size(0))
        top1.update(prec.item(), input.size(0))

        # compute gradient and do SGD step
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # measure elapsed time
        batch_time.update(time.time() - end) # time spent to process one batch
        end = time.time()


        if i % print_freq == 0:
            print('Epoch: [{0}][{1}/{2}]\t'
                  'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
                  'Data {data_time.val:.3f} ({data_time.avg:.3f})\t'
                  'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
                  'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                   epoch, i, len(trainloader), batch_time=batch_time,
                   data_time=data_time, loss=losses, top1=top1))



def validate(val_loader, model, criterion ):
    batch_time = AverageMeter()
    losses = AverageMeter()
    top1 = AverageMeter()

    # switch to evaluate mode
    model.eval()
```

```
    end = time.time()
    with torch.no_grad():
        for i, (input, target) in enumerate(val_loader):

            input, target = input.to(device), target.to(device)

            # compute output
            output = model(input)
            loss = criterion(output, target)

            # measure accuracy and record loss
            prec = accuracy(output, target)[0]
            losses.update(loss.item(), input.size(0))
            top1.update(prec.item(), input.size(0))

            # measure elapsed time
            batch_time.update(time.time() - end)
            end = time.time()

            if i % print_freq == 0:  # This line shows how frequently print out
                print('Test: [{0}/{1}]\t'
                  'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
                  'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
                  'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                   i, len(val_loader), batch_time=batch_time, loss=losses,
                   top1=top1))

    print(' * Prec {top1.avg:.3f}% '.format(top1=top1))
    return top1.avg


def accuracy(output, target, topk=(1,5)):
    """Computes the precision@k for the specified values of k"""
    maxk = max(topk) # 5
    batch_size = target.size(0) # 128

    _, pred = output.topk(maxk, 1, True, True) # topk(k, dim=None, largest=True,
                                    # will output (max value, its index)
    pred = pred.t()                # transpose
    correct = pred.eq(target.view(1, -1).expand_as(pred))   # "-1": calculate au

    res = []
    for k in topk: # 1, 5
        correct_k = correct[:k].reshape(-1).float().sum(0)  # view(-1): make a f
        res.append(correct_k.mul_(100.0 / batch_size))   # correct: size of [max
    return res


class AverageMeter(object):
    """Computes and stores the average and current value"""
    def __init__(self):
        self.reset()

    def reset(self):
        self.val = 0
        self.avg = 0
        self.sum = 0
        self.count = 0
```

```
    def update(self, val, n=1):
        self.val = val
        self.sum += val * n    ## n is impact factor
        self.count += n
        self.avg = self.sum / self.count


def save_checkpoint(state, is_best, fdir):
    filepath = os.path.join(fdir, 'checkpoint.pth')
    torch.save(state, filepath)
    if is_best:
        shutil.copyfile(filepath, os.path.join(fdir, 'model_best.pth.tar'))


def adjust_learning_rate(optimizer, epoch):
    """For resnet, the lr starts from 0.1, and is divided by 10 at 80 and 120 ep
    adjust_list = [150, 225]
    if epoch in adjust_list:
        for param_group in optimizer.param_groups:
            param_group['lr'] = param_group['lr'] * 0.1

#model = nn.DataParallel(model).cuda()
#all_params = checkpoint['state_dict']
#model.load_state_dict(all_params, strict=False)
#criterion = nn.CrossEntropyLoss().cuda()
#validate(testloader, model, criterion)
```

```
=> Building model...
Files already downloaded and verified
Files already downloaded and verified
```

In [81]:

```
import matplotlib.pyplot as plt
import numpy as np

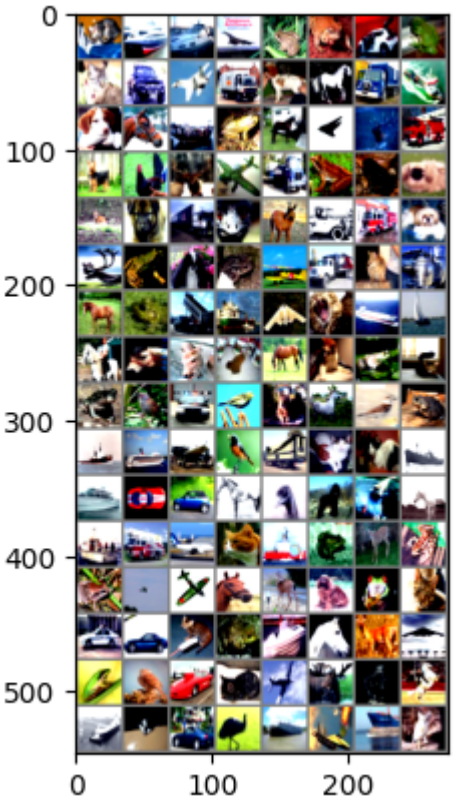
# functions to show an image

def imshow(img):
    img = img / 2 + 0.5     # unnormalize
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()

# get some random training images
dataiter = iter(testloader)
images, labels = next(dataiter) ## If you run this line, the next data batch is

# show images
imshow(torchvision.utils.make_grid(images))
```

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for float
s or [0..255] for integers). Got range [-0.49392712..1.5658436].
```



In [ ]:
```
# Training settings
lr = 0.1
weight_decay = 5e-4
epochs = 200
best_prec = 0

model = model.to(device)
criterion = nn.CrossEntropyLoss().to(device)
optimizer = torch.optim.SGD(
    model.parameters(), lr=lr, momentum=0.9,
    weight_decay=weight_decay, nesterov=True
)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs)

if not os.path.exists('result'):
    os.makedirs('result')

fdir = 'result/' + str(model_name)
os.makedirs(fdir, exist_ok=True)

for epoch in range(0, epochs):
    # adjust_learning_rate(optimizer, epoch)

    train(trainloader, model, criterion, optimizer, epoch)

    # evaluate on test set
    print("Validation starts")
    prec = validate(testloader, model, criterion)

    # remember best precision and save checkpoint
    is_best = prec > best_prec
    best_prec = max(prec, best_prec)
    print('best acc: {:1f}'.format(best_prec))
    save_checkpoint({
```

```
        'epoch': epoch + 1,
        'state_dict': model.state_dict(),
        'best_prec': best_prec,
        'optimizer': optimizer.state_dict(),
    }, is_best, fdir)

    scheduler.step()
    print('Learning rate: {:.6f}'.format(optimizer.param_groups[0]['lr']))
```

In [82]:
```
# Reload the saved best checkpoint and evaluate it
fdir = 'result/' + str(model_name) + '/model_best.pth.tar'
checkpoint = torch.load(fdir, map_location=device)
model.load_state_dict(checkpoint['state_dict'])


criterion = nn.CrossEntropyLoss().to(device)

model.eval()
model.to(device)


prec = validate(testloader, model, criterion)
print('Final test accuracy: {:.2f}%'.format(prec))
```

```
Test: [0/79]    Time 6.152 (6.152)      Loss 0.1314 (0.1314)    Prec 92.969% (92.
969%)
 * Prec 92.790%
Final test accuracy: 92.79%
```

HW4 — ResNet-20 Post-Training Weight Quantization

The baseline accuracy above is measured using the original trained checkpoint. The next two cells perform weight-only post-training quantization (PTQ) at 8 bits and 4 bits.

Alpha choice: each `Conv2d` layer uses its own `alpha = scale_pct*max(abs(weight))`.

Scale choice: we run a **MSE** based search over 0.4 - 1.0 with 1000 candidates for each layer.

Run the baseline cell and then run the 8-bit and 4-bit cells. The test accuracy printed by each cell can be compared with the baseline accuracy.

In [86]:
```
# Code8-style weight quantization helper
def quantize_weight(weight, w_bit, alpha):
    # self.w_bit = w_bit - 1, then quantize magnitudes with 2**self.w_bit - 1.
    b = w_bit - 1

    if alpha.item() == 0:
        return weight.clone()

    input_d = weight / alpha
    input_c = input_d.clamp(min=-1, max=1)
    sign = input_c.sign()
    input_abs = input_c.abs()
```

```
    levels = 2 ** b - 1
    input_q = input_abs.mul(levels).round().div(levels).mul(sign)

    return input_q.mul(alpha)

def find_optimal_alpha_mse(weights, w_bit):
    max_val = weights.abs().max()

    best_alpha = max_val
    best_mse = float('inf')

    # Test 1000 candidate alphas between 40% and 100% of max value
    for scale_pct in torch.linspace(0.4, 1.0, 1000):
        candidate_alpha = max_val * scale_pct
        dequantized_w = quantize_weight(weights, w_bit, candidate_alpha)
        mse = torch.mean((weights - dequantized_w) ** 2).item()

        if mse < best_mse:
            best_mse = mse
            best_alpha = candidate_alpha

    return best_alpha

def quantize_conv_weights(model, w_bit, use_mse_search = True):
    quantized_layers = 0
    with torch.no_grad():
        for layer in model.modules():
            if isinstance(layer, torch.nn.Conv2d):
                # Per-Layer alpha/clipping value.
                max = layer.weight.detach().abs().max()

                if use_mse_search:
                    alpha = find_optimal_alpha_mse(layer.weight,w_bit)
                    print(f"Layer {quantized_layers+1} alpha choosen: {alpha.ite

                    layer.weight.copy_(quantize_weight(layer.weight, w_bit, alph
                else:
                    layer.weight.copy_(quantize_weight(layer.weight, w_bit, max)

                quantized_layers += 1
    return quantized_layers
```

8-bit weight PTQ (with MSE Search for alpha)

In [87]:
```
# Reload the original trained checkpoint, then apply 8-bit weight PTQ.
checkpoint_path = os.path.join("result", str(model_name), "model_best.pth.tar")
checkpoint = torch.load(checkpoint_path, map_location=device)
model.load_state_dict(checkpoint["state_dict"])
model.to(device)
model.eval()

num_quantized_layers = quantize_conv_weights(model, w_bit=8)

criterion = nn.CrossEntropyLoss().to(device)
print(f"Applied 8-bit PTQ to {num_quantized_layers} Conv2d layers.")
acc_8bit = validate(testloader, model, criterion)
```

```
print("Final test accuracy after 8-bit weight PTQ: {:.2f}%".format(acc_8bit))
print("Accuracy change vs. original baseline: {:+.2f} percentage points".format(
```

```
Layer 1 alpha choosen: 0.7746 (Clipped at 99.82% of max)
Layer 2 alpha choosen: 0.4591 (Clipped at 98.86% of max)
Layer 3 alpha choosen: 0.2700 (Clipped at 98.62% of max)
Layer 4 alpha choosen: 0.2509 (Clipped at 100.00% of max)
Layer 5 alpha choosen: 0.3169 (Clipped at 99.94% of max)
Layer 6 alpha choosen: 0.2468 (Clipped at 100.00% of max)
Layer 7 alpha choosen: 0.2927 (Clipped at 99.82% of max)
Layer 8 alpha choosen: 0.2418 (Clipped at 97.54% of max)
Layer 9 alpha choosen: 0.3081 (Clipped at 95.68% of max)
Layer 10 alpha choosen: 0.5136 (Clipped at 99.88% of max)
Layer 11 alpha choosen: 0.2223 (Clipped at 95.98% of max)
Layer 12 alpha choosen: 0.2135 (Clipped at 96.04% of max)
Layer 13 alpha choosen: 0.2233 (Clipped at 98.08% of max)
Layer 14 alpha choosen: 0.2267 (Clipped at 97.24% of max)
Layer 15 alpha choosen: 0.1960 (Clipped at 94.59% of max)
Layer 16 alpha choosen: 0.1782 (Clipped at 92.73% of max)
Layer 17 alpha choosen: 0.2469 (Clipped at 98.62% of max)
Layer 18 alpha choosen: 0.1958 (Clipped at 88.41% of max)
Layer 19 alpha choosen: 0.1639 (Clipped at 84.86% of max)
Layer 20 alpha choosen: 0.1496 (Clipped at 91.11% of max)
Layer 21 alpha choosen: 0.0994 (Clipped at 89.25% of max)
Applied 8-bit PTQ to 21 Conv2d layers.
Test: [0/79]    Time 5.065 (5.065)      Loss 0.1303 (0.1303)    Prec 92.969% (92.
969%)
 * Prec 92.830%
Final test accuracy after 8-bit weight PTQ: 92.83%
Accuracy change vs. original baseline: +0.04 percentage points
```

4-bit weight PTQ (with MSE Search for alpha)

In [88]:
```
# Reload the original trained checkpoint again, then apply 4-bit weight PTQ.
checkpoint_path = os.path.join("result", str(model_name), "model_best.pth.tar")
checkpoint = torch.load(checkpoint_path, map_location=device)
model.load_state_dict(checkpoint["state_dict"])
model.to(device)
model.eval()

num_quantized_layers = quantize_conv_weights(model, w_bit=4)

criterion = nn.CrossEntropyLoss().to(device)
print(f"Applied 4-bit PTQ to {num_quantized_layers} Conv2d layers.")
acc_4bit = validate(testloader, model, criterion)
print("Final test accuracy after 4-bit weight PTQ: {:.2f}%".format(acc_4bit))
print("Accuracy change vs. original baseline: {:+.2f} percentage points".format(
```

```
Layer 1 alpha choosen: 0.5415 (Clipped at 69.79% of max)
Layer 2 alpha choosen: 0.2234 (Clipped at 48.11% of max)
Layer 3 alpha choosen: 0.1925 (Clipped at 70.33% of max)
Layer 4 alpha choosen: 0.1686 (Clipped at 67.21% of max)
Layer 5 alpha choosen: 0.1647 (Clipped at 51.95% of max)
Layer 6 alpha choosen: 0.1850 (Clipped at 74.95% of max)
Layer 7 alpha choosen: 0.1737 (Clipped at 59.22% of max)
Layer 8 alpha choosen: 0.1484 (Clipped at 59.88% of max)
Layer 9 alpha choosen: 0.1375 (Clipped at 42.70% of max)
Layer 10 alpha choosen: 0.3249 (Clipped at 63.18% of max)
Layer 11 alpha choosen: 0.1206 (Clipped at 52.07% of max)
Layer 12 alpha choosen: 0.0993 (Clipped at 44.68% of max)
Layer 13 alpha choosen: 0.1268 (Clipped at 55.68% of max)
Layer 14 alpha choosen: 0.0940 (Clipped at 40.30% of max)
Layer 15 alpha choosen: 0.1100 (Clipped at 53.09% of max)
Layer 16 alpha choosen: 0.0982 (Clipped at 51.11% of max)
Layer 17 alpha choosen: 0.1466 (Clipped at 58.56% of max)
Layer 18 alpha choosen: 0.0963 (Clipped at 43.48% of max)
Layer 19 alpha choosen: 0.0848 (Clipped at 43.90% of max)
Layer 20 alpha choosen: 0.0867 (Clipped at 52.79% of max)
Layer 21 alpha choosen: 0.0524 (Clipped at 47.09% of max)
Applied 4-bit PTQ to 21 Conv2d layers.
Test: [0/79]    Time 4.755 (4.755)      Loss 0.3335 (0.3335)    Prec 86.719% (86.
719%)
 * Prec 89.190%
Final test accuracy after 4-bit weight PTQ: 89.19%
Accuracy change vs. original baseline: -3.60 percentage points
```

8-bit weight PTQ (without MSE Search for alpha)

In [ ]:
```
# Reload the original trained checkpoint, then apply 8-bit weight PTQ.
checkpoint_path = os.path.join("result", str(model_name), "model_best.pth.tar")
checkpoint = torch.load(checkpoint_path, map_location=device)
model.load_state_dict(checkpoint["state_dict"])
model.to(device)
model.eval()

num_quantized_layers = quantize_conv_weights(model, w_bit=8 , use_mse_search=Fal

criterion = nn.CrossEntropyLoss().to(device)
print(f"Applied 8-bit PTQ to {num_quantized_layers} Conv2d layers (without MSE a
acc_8bit = validate(testloader, model, criterion)
print("Final test accuracy after 8-bit weight PTQ: {:.2f}%".format(acc_8bit))
print("Accuracy change vs. original baseline: {:+.2f} percentage points".format(
```

```
Applied 8-bit PTQ to without MSE alpha search 21 Conv2d layers.
Test: [0/79]    Time 4.895 (4.895)      Loss 0.1292 (0.1292)    Prec 92.969% (92.
969%)
 * Prec 92.820%
Final test accuracy after 8-bit weight PTQ: 92.82%
Accuracy change vs. original baseline: +0.03 percentage points
```

4-bit weight PTQ (without MSE Search for alpha)

In [92]:
```
# Reload the original trained checkpoint again, then apply 4-bit weight PTQ.
checkpoint_path = os.path.join("result", str(model_name), "model_best.pth.tar")
checkpoint = torch.load(checkpoint_path, map_location=device)
model.load_state_dict(checkpoint["state_dict"])
```

```
model.to(device)
model.eval()

num_quantized_layers = quantize_conv_weights(model, w_bit=4, use_mse_search=Fals

criterion = nn.CrossEntropyLoss().to(device)
print(f"Applied 4-bit PTQ to {num_quantized_layers} Conv2d layers (without MSE a
acc_4bit = validate(testloader, model, criterion)
print("Final test accuracy after 4-bit weight PTQ: {:.2f}%".format(acc_4bit))
print("Accuracy change vs. original baseline: {:+.2f} percentage points".format(
```

```
Applied 4-bit PTQ to 21 Conv2d layers (without MSE alpha search).
Test: [0/79]    Time 4.979 (4.979)      Loss 0.3682 (0.3682)    Prec 87.500% (87.
500%)
 * Prec 87.730%
Final test accuracy after 4-bit weight PTQ: 87.73%
Accuracy change vs. original baseline: -5.06 percentage points
```

HW 4

Name: Chaitany Raghav
TSN: 200478852

Q1.3

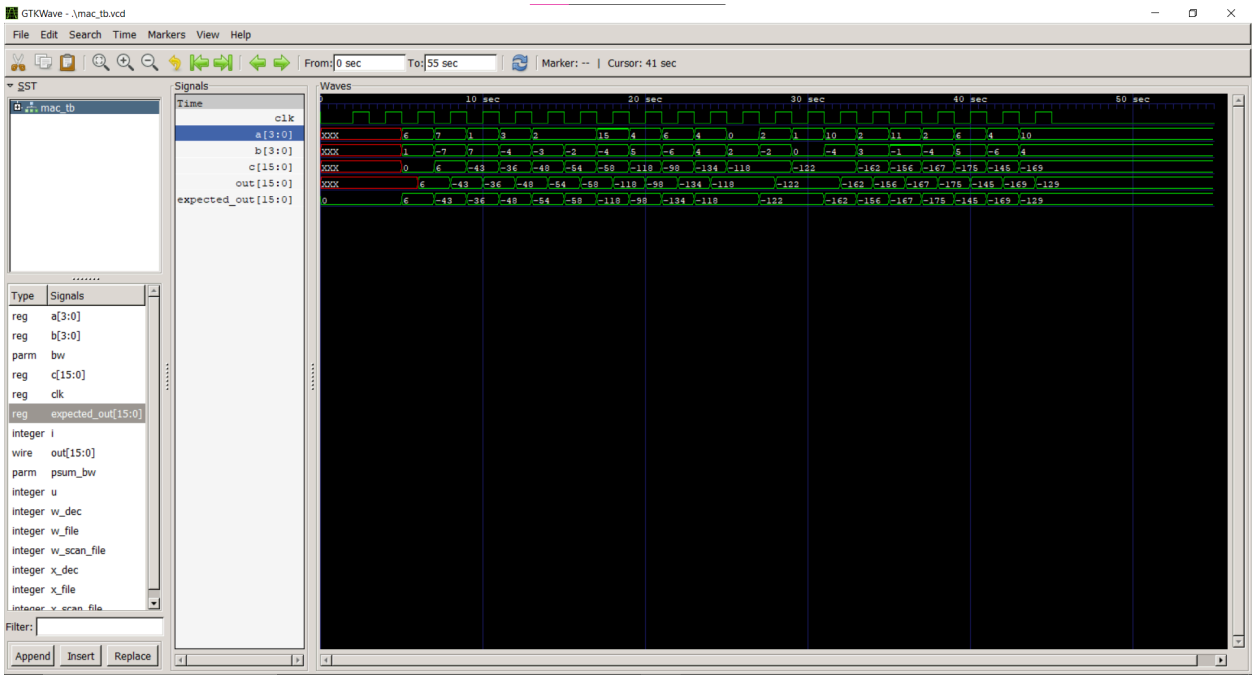
Observations:

Model	Top - 1 Accuracy	Δ Accuracy
Baseline	92.79	-
8 Bit PQT	92.82	+0.03
8 Bit PQT with MSE-based α search	92.83	+0.04
4 Bit PQT	87.73	-5.06
4 Bit PQT with MSE-based α search	89.19	-3.60

- **8-Bit Weight Quantization (+0.04%):** Compressing the weights to 8 bits gives the model 256 discrete levels to work with. This resolution is more than enough to capture all the core features of the network. Interestingly, the slight rounding effect actually acts like a mild regularization filter. Smoothing out the tiny, noisy decimal variations in the original FP32 baseline helps reduce minor overfitting and allows the model to generalize slightly better on the test set.
- **4-Bit Weight Quantization (-3.60%):** Squeezing everything into 4 bits drops the resolution down to just 16 discrete levels. This harsh compression introduces significant quantization noise and creates an outlier issue. Without optimization, a few highly sensitive weight or activation spikes force the quantization scale to stretch too wide. This ends up crushing 99% of the normal weights into the same few bins, which completely strips away the model's ability to handle fine-grained details and causes a drop in accuracy.
- **The Alpha Strategy:** Implementing an alpha-parameter search essentially gives us a smart mathematical slider. It dynamically scales down the extreme outliers before the model gets quantized. For the 8-bit model, it fine-tuned an already stable setup to maximize the regularization benefits. For the 4-bit model, it was a lifesaver—protecting the most critical 1% of features from being wiped out entirely. This alone salvaged a massive chunk of performance, clawing back 1.46% of baseline accuracy to bring the drop down from -5.06% to a much more acceptable -3.60%.

Q 2

Waveform:



mac.v:

```
// Created by prof. Mingu Kang @VVIP Lab in UCSD ECE department
// Please do not spread this code without permission
module mac (out, a, b, c);

parameter bw = 4;
parameter psum_bw = 16;

input unsigned [bw-1:0] a;
input signed [bw-1:0] b;

input signed [psum_bw-1:0] c;

output signed [psum_bw-1:0] out;

wire signed [bw:0] a_signed;
wire signed [2*bw:0] product;
```

```
assign a_signed = $signed({1'b0, a});
assign product = a_signed * b;
assign out = c + product;

endmodule
```

mac_tb.v:

```
// Created by prof. Mingu Kang @VVIP Lab in UCSD ECE department
// Please do not spread this code without permission


module mac_tb;

parameter bw = 4;
parameter psum_bw = 16;

reg clk = 0;

reg  [bw-1:0] a;
reg  [bw-1:0] b;
reg  [psum_bw-1:0] c;
wire [psum_bw-1:0] out;
reg  [psum_bw-1:0] expected_out = 0;

integer w_file ; // file handler
integer w_scan_file ; // file handler

integer x_file ; // file handler
integer x_scan_file ; // file handler

integer x_dec;
integer w_dec;
integer i;
integer u;

function [3:0] w_bin ;
  input integer  weight ;
  begin
```

```
    if (weight>-1)
     w_bin[3] = 0;
    else begin
     w_bin[3] = 1;
     weight = weight + 8;
    end

    if (weight>3) begin
     w_bin[2] = 1;
     weight = weight - 4;
    end
    else
     w_bin[2] = 0;

    if (weight>1) begin
     w_bin[1] = 1;
     weight = weight - 2;
    end
    else
     w_bin[1] = 0;

    if (weight>0)
     w_bin[0] = 1;
    else
     w_bin[0] = 0;

  end
endfunction



function [3:0] x_bin ;
  input integer  weight ;
  begin
    for (u=bw-1; u>-1; u=u-1) begin
      if (weight>2**u-1) begin
        x_bin[u] = 1;
        weight = weight - 2**u;
      end
      else begin
```

```
        x_bin[u] = 0;
      end
    end
  end
endfunction


// Below function is for verification
function [psum_bw-1:0] mac_predicted;
    input [bw-1:0] a;
    input signed [bw-1:0] b;
    input signed [psum_bw-1:0] c;

    reg signed [bw:0] a_signed;
    reg signed [2*bw:0] product;

    begin
        a_signed = $signed({1'b0, a});
        product = a_signed * b;
        mac_predicted = c + product;
    end
endfunction



mac_wrapper #(.bw(bw), .psum_bw(psum_bw)) mac_wrapper_instance (
  .clk(clk),
        .a(a),
        .b(b),
        .c(c),
  .out(out)
);


initial begin

  w_file = $fopen("b_data.txt", "r");  //weight data
  x_file = $fopen("a_data.txt", "r");  //activation

  $dumpfile("mac_tb.vcd");
```

```
  $dumpvars(0,mac_tb);

  #1 clk = 1'b0;
  #1 clk = 1'b1;
  #1 clk = 1'b0;

  $display("-------------------- Computation start --------------------");


  for (i=0; i<20; i=i+1) begin  // Data lenght is 10 in the data files

     #1 clk = 1'b1;
     #1 clk = 1'b0;

     w_scan_file = $fscanf(w_file, "%d\n", w_dec);
     x_scan_file = $fscanf(x_file, "%d\n", x_dec);

     a = x_bin(x_dec); // unsigned number
     b = w_bin(w_dec); // signed number
     c = expected_out;

     expected_out = mac_predicted(a, b, c);

  end



  #1 clk = 1'b1;
  #1 clk = 1'b0;

  $display("-------------------- Computation completed
--------------------");

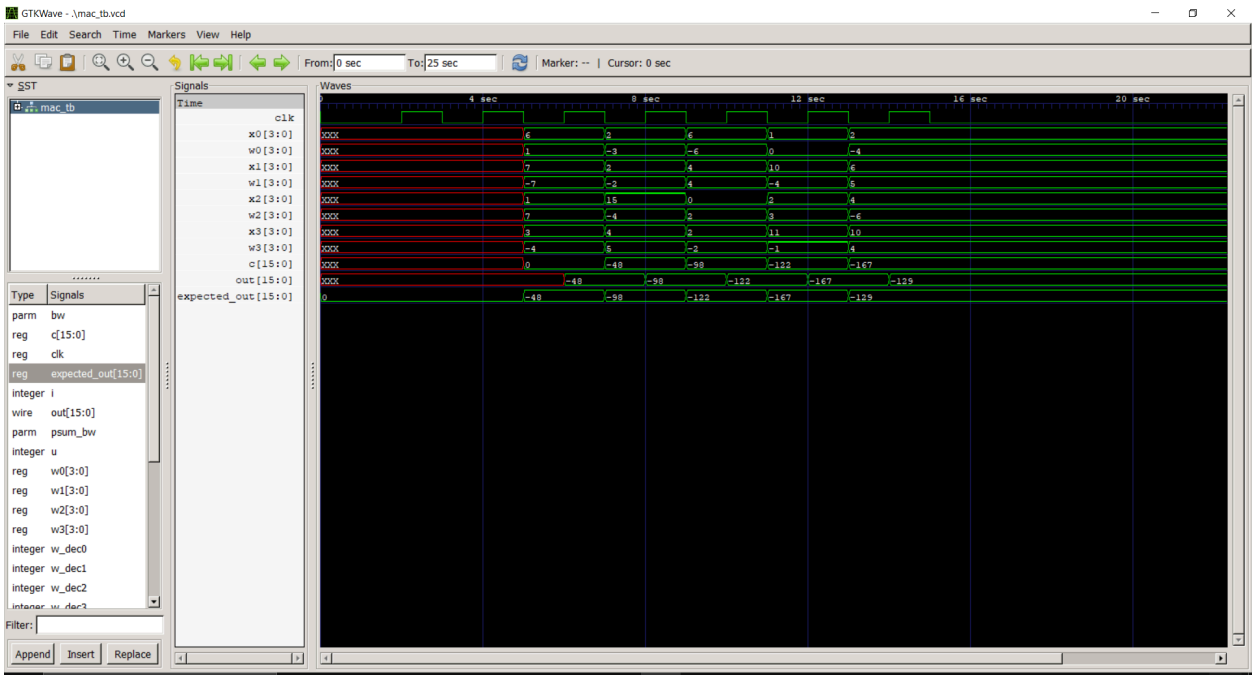
  #10 $finish;


end

endmodule
```

Q 3

Waveform:



mac.v:

```
// Created by prof. Mingu Kang @VVIP Lab in UCSD ECE department
// Please do not spread this code without permission
module mac (out, x0, w0, x1, w1, x2, w2, x3, w3, in);

parameter bw = 4;
parameter psum_bw = 16;

input unsigned [bw-1:0] x0;
input unsigned [bw-1:0] x1;
input unsigned [bw-1:0] x2;
input unsigned [bw-1:0] x3;
input signed [bw-1:0] w0;
input signed [bw-1:0] w1;
input signed [bw-1:0] w2;
input signed [bw-1:0] w3;

input signed [psum_bw-1:0] in;
```

```
output signed [psum_bw-1:0] out;

wire signed [bw:0] x0_signed;
wire signed [bw:0] x1_signed;
wire signed [bw:0] x2_signed;
wire signed [bw:0] x3_signed;
wire signed [2*bw:0] product0;
wire signed [2*bw:0] product1;
wire signed [2*bw:0] product2;
wire signed [2*bw:0] product3;

wire signed [2*bw+1:0] internal_sum0;
wire signed [2*bw+1:0] internal_sum1;

assign x0_signed = $signed({1'b0, x0});
assign x1_signed = $signed({1'b0, x1});
assign x2_signed = $signed({1'b0, x2});
assign x3_signed = $signed({1'b0, x3});

assign product0 = x0_signed * w0;
assign product1 = x1_signed * w1;
assign product2 = x2_signed * w2;
assign product3 = x3_signed * w3;

assign internal_sum0 = product0 + product1;
assign internal_sum1 = product2 + product3;

assign out = internal_sum0 + internal_sum1 + in;

endmodule
```

mac_wrapper.v:

```
// Created by prof. Mingu Kang @VVIP Lab in UCSD ECE department
// Please do not spread this code without permission
module mac_wrapper (
    clk,
    out,
    x0, x1, x2, x3,
    w0, w1, w2, w3,
```

```
    c
);

parameter bw = 4;
parameter psum_bw = 16;

output [psum_bw-1:0] out;
input  [bw-1:0] x0;
input  [bw-1:0] x1;
input  [bw-1:0] x2;
input  [bw-1:0] x3;
input  [bw-1:0] w0;
input  [bw-1:0] w1;
input  [bw-1:0] w2;
input  [bw-1:0] w3;
input  [psum_bw-1:0] c;
input  clk;

// Separate pipeline registers for each input channel
reg    [bw-1:0] x0_q, x1_q, x2_q, x3_q;
reg    [bw-1:0] w0_q, w1_q, w2_q, w3_q;
reg    [psum_bw-1:0] c_q;

// Structural instantiation matching the new mac design explicitly
mac #(.bw(bw), .psum_bw(psum_bw)) mac_instance (
        .x0(x0_q),
        .w0(w0_q),
        .x1(x1_q),
        .w1(w1_q),
        .x2(x2_q),
        .w2(w2_q),
        .x3(x3_q),
        .w3(w3_q),
        .in(c_q),
        .out(out)
);

always @ (posedge clk) begin
        x0_q <= x0;
        x1_q <= x1;
```

```
        x2_q <= x2;
        x3_q <= x3;

        w0_q <= w0;
        w1_q <= w1;
        w2_q <= w2;
        w3_q <= w3;

        c_q  <= c;
end

endmodule
```

mac_tb.v:

```
// Created by prof. Mingu Kang @VVIP Lab in UCSD ECE department
// Please do not spread this code without permission


module mac_tb;

parameter bw = 4;
parameter psum_bw = 16;

reg clk = 0;

reg  [bw-1:0] x0, x1, x2, x3;
reg  [bw-1:0] w0, w1, w2, w3;
reg  [psum_bw-1:0] c;
wire [psum_bw-1:0] out;
reg  [psum_bw-1:0] expected_out = 0;

integer w_file ; // file handler
integer w_scan_file ; // file handler

integer x_file ; // file handler
integer x_scan_file ; // file handler

integer x_dec0, x_dec1, x_dec2, x_dec3;
integer w_dec0, w_dec1, w_dec2, w_dec3;
```

```
integer i;
integer u;

function [3:0] w_bin ;
  input integer  weight ;
  begin

    if (weight>-1)
     w_bin[3] = 0;
    else begin
     w_bin[3] = 1;
     weight = weight + 8;
    end

    if (weight>3) begin
     w_bin[2] = 1;
     weight = weight - 4;
    end
    else
     w_bin[2] = 0;

    if (weight>1) begin
     w_bin[1] = 1;
     weight = weight - 2;
    end
    else
     w_bin[1] = 0;

    if (weight>0)
     w_bin[0] = 1;
    else
     w_bin[0] = 0;

  end
endfunction



function [3:0] x_bin ;
  input integer  weight ;
```

```
  begin
    for (u=bw-1; u>-1; u=u-1) begin
      if (weight>2**u-1) begin
        x_bin[u] = 1;
        weight = weight - 2**u;
      end
      else begin
        x_bin[u] = 0;
      end
    end
  end
endfunction

// Below function is for verification
function [psum_bw-1:0] mac_predicted;
    input [bw-1:0] x0;
    input [bw-1:0] x1;
    input [bw-1:0] x2;
    input [bw-1:0] x3;
    input signed [bw-1:0] w0;
    input signed [bw-1:0] w1;
    input signed [bw-1:0] w2;
    input signed [bw-1:0] w3;
    input signed [psum_bw-1:0] acc_in;

    reg signed [bw:0] x0_s, x1_s, x2_s, x3_s;
    reg signed [2*bw:0] p0, p1, p2, p3;

    begin
        x0_s = $signed({1'b0, x0});
        x1_s = $signed({1'b0, x1});
        x2_s = $signed({1'b0, x2});
        x3_s = $signed({1'b0, x3});

        p0 = x0_s * w0;
        p1 = x1_s * w1;
        p2 = x2_s * w2;
        p3 = x3_s * w3;

        mac_predicted = acc_in + p0 + p1 + p2 + p3;
```

```
    end
endfunction


mac_wrapper #(.bw(bw), .psum_bw(psum_bw)) mac_wrapper_instance (
        .clk(clk),
        .x0(x0), .x1(x1), .x2(x2), .x3(x3),
        .w0(w0), .w1(w1), .w2(w2), .w3(w3),
        .c(c),
        .out(out)
);


initial begin
  w_file = $fopen("b_data.txt", "r");  //weight data
  x_file = $fopen("a_data.txt", "r");  //activation

  $dumpfile("mac_tb.vcd");
  $dumpvars(0,mac_tb);

  #1 clk = 1'b0;
  #1 clk = 1'b1;
  #1 clk = 1'b0;

  $display("-------------------- Computation start --------------------");


  for (i=0; i<5; i=i+1) begin

    #1 clk = 1'b1;
    #1 clk = 1'b0;

    w_scan_file = $fscanf(w_file, "%d\n", w_dec0);
    w_scan_file = $fscanf(w_file, "%d\n", w_dec1);
    w_scan_file = $fscanf(w_file, "%d\n", w_dec2);
    w_scan_file = $fscanf(w_file, "%d\n", w_dec3);

    x_scan_file = $fscanf(x_file, "%d\n", x_dec0);
    x_scan_file = $fscanf(x_file, "%d\n", x_dec1);
    x_scan_file = $fscanf(x_file, "%d\n", x_dec2);
```

```
    x_scan_file = $fscanf(x_file, "%d\n", x_dec3);

    x0 = x_bin(x_dec0);
    x1 = x_bin(x_dec1);
    x2 = x_bin(x_dec2);
    x3 = x_bin(x_dec3);

    w0 = w_bin(w_dec0);
    w1 = w_bin(w_dec1);
    w2 = w_bin(w_dec2);
    w3 = w_bin(w_dec3);

    c = expected_out;

    expected_out = mac_predicted(x0, x1, x2, x3, w0, w1, w2, w3, c);

  end



  #1 clk = 1'b1;
  #1 clk = 1'b0;

  $display("--------------------- Computation completed
---------------------");

  #10 $finish;


end

endmodule
```

UCSD Linux Cloud: N/A. This project was done on a local setup.
Github: https://github.com/chaitany-raghav/ece284fa26/tree/my-solutions